

CS570 Week 3 Lab: Analyzing Spotify Data with Spark

How this lab works:

- Your instructor will walk through the code in Parts 1-7
- Follow along and run each cell on your laptop
- Answer all Questions as you go (or after class)
- Complete Part 8 on your own.

Objectives:

- Understand RDD partitioning and parallelism
- Practice transformations vs actions
- Experience lazy evaluation firsthand
- Use Pair RDDs for aggregations
- Observe the benefits of caching

Dataset: 114,000 Spotify tracks with audio features like danceability, energy, and popularity.

Part 1: Starting Spark

We'll use `SparkSession` — the unified entry point for modern Spark. It gives us both `DataFrame` and `RDD` capabilities.

```
In [1]: import os
import sys

# Ensure Spark workers use the same Python as this notebook
# (Fixes version mismatch errors when using virtual environments)
os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

from pyspark.sql import SparkSession
import time

# Create SparkSession with all available cores
spark = SparkSession.builder \
    .appName("CS570 Spotify Analysis") \
    .master("local[*]") \
    .getOrCreate()

# Get SparkContext from session (for RDD operations)
sc = spark.sparkContext
```

```
print(f"Spark version: {spark.version}")
print(f"Python version: {sys.version.split()[0]}")
print(f"Master: {sc.master}")
print(f"Default parallelism (cores): {sc.defaultParallelism}")

sc.setLogLevel("ERROR") # Suppress warnings for cleaner output

print(f"\n✓ Spark is ready with {sc.defaultParallelism} cores!")
```

Spark version: 3.5.3
 Python version: 3.11.9
 Master: local[*]
 Default parallelism (cores): 16

✓ Spark is ready with 16 cores!

Question 1

What does `local[*]` mean? What would `local[2]` do instead?

Double-click to edit and write your answer below:

Your Answer:

`local[*]` means Spark runs locally using all available CPU cores — the `*` auto-detects every core on the machine. `local[2]` would restrict Spark to only 2 cores, meaning only 2 tasks can run in parallel regardless of how many cores the machine actually has.

Part 2: Loading the Data

We'll download a Spotify dataset and load it the simple way first. This might work... or it might not!

```
In [2]: # Download the dataset (only need to run once)
import urllib.request
import os

data_path = "spotify.csv"

if not os.path.exists(data_path):
    print("Downloading Spotify dataset...")
    url = "https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset/re
    urllib.request.urlretrieve(url, data_path)
    print(f"✓ Downloaded to {data_path}")
else:
    print(f"✓ Dataset already exists at {data_path}")

# Check file size
size_mb = os.path.getsize(data_path) / (1024 * 1024)
print(f"File size: {size_mb:.1f} MB")
```

✓ Dataset already exists at spotify.csv
File size: 19.2 MB

```
In [3]: # Load CSV – the simple way
from pyspark.sql.functions import col

df_raw = spark.read.csv(data_path, header=True, inferSchema=True)

print(f"Rows: {df_raw.count():,}")
print(f"Columns: {len(df_raw.columns)}")
```

Rows: 114,000
Columns: 21

```
In [4]: df_raw.printSchema()
```

```
root
|-- _c0: integer (nullable = true)
|-- track_id: string (nullable = true)
|-- artists: string (nullable = true)
|-- album_name: string (nullable = true)
|-- track_name: string (nullable = true)
|-- popularity: string (nullable = true)
|-- duration_ms: string (nullable = true)
|-- explicit: string (nullable = true)
|-- danceability: string (nullable = true)
|-- energy: string (nullable = true)
|-- key: string (nullable = true)
|-- loudness: string (nullable = true)
|-- mode: string (nullable = true)
|-- speechiness: string (nullable = true)
|-- acousticness: string (nullable = true)
|-- instrumentalness: double (nullable = true)
|-- liveness: string (nullable = true)
|-- valence: string (nullable = true)
|-- tempo: double (nullable = true)
|-- time_signature: double (nullable = true)
|-- track_genre: string (nullable = true)
```

```
In [5]: # Preview a few rows
df_raw.select("track_name", "artists", "track_genre", "popularity", "danceability",
```

```

+-----+-----+-----+-----+-----+
--+-----+-----+
|          track_name|          artists|track_genre|popularity|danceabili
ty|energy|valence|
+-----+-----+-----+-----+-----+
--+-----+-----+
|          Comedy|          Gen Hoshino|  acoustic|      73|      0.6
76| 0.461| 0.715|
|          Ghost - Acoustic|          Ben Woodward|  acoustic|      55|      0.
42| 0.166| 0.267|
|          To Begin Again|Ingrid Michaelson;ZAYN|  acoustic|      57|      0.4
38| 0.359| 0.12|
|Can't Help Falling In Love|          Kina Grannis|  acoustic|      71|      0.2
66|0.0596| 0.143|
|          Hold On|          Chord Overstreet|  acoustic|      82|      0.6
18| 0.443| 0.167|
+-----+-----+-----+-----+-----+
--+-----+-----+
only showing top 5 rows

```

2.1 Converting to RDD

Now let's convert to RDD and access the data. Let's see what happens...

```

In [7]: # Convert DataFrame to RDD
tracks_rdd = df_raw.rdd

print(f"Type: {type(tracks_rdd)}")
print(f"Number of partitions: {tracks_rdd.getNumPartitions()}")
print(f"Number of cores: {sc.defaultParallelism}")

# Look at one row
sample = tracks_rdd.first()
print(f"\nSample row type: {type(sample)}")
print(f"\nAccess by name:")
print(f"  Track: {sample.track_name}")
print(f"  Artist: {sample.artists}")
print(f"  Genre: {sample.track_genre}")
print(f"  Energy: {sample.energy}")

```

Type: <class 'pyspark.rdd.RDD'>

Number of partitions: 5

Number of cores: 16

Sample row type: <class 'pyspark.sql.types.Row'>

Access by name:

Track: Comedy

Artist: Gen Hoshino

Genre: acoustic

Energy: 0.461

 Did You Get an Error?

Welcome to real-world data! This dataset has messy rows where:

- Track names contain commas (e.g., "Hello, World")
- Simple CSV parsing breaks on these → columns shift → numbers become strings

Reading the Error Log

Spark error messages are verbose, but the key info is there:

1. **Find the error type:** `CAST_INVALID_INPUT` — a type conversion failed
2. **Find the bad value:** `' Pt. 1) [Music from the Original TV Series]'` — clearly not a number!
3. **Understand the cause:** A quoted field with commas caused column misalignment

The Fix

We need to tell Spark how to handle quoted fields properly. Also we shouldn't always trust the system to infer the schema for us. Discuss which fields you would import differently. Run the cell below to reload the data with the correct options.

```
In [8]: # Reload with proper CSV options
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("quote", '"') \
    .option("escape", '\\') \
    .option("multiline", "true") \
    .csv(data_path)

# Cast numeric columns
df = df.withColumn("popularity", col("popularity").cast("int")) \
    .withColumn("energy", col("energy").cast("double")) \
    .withColumn("danceability", col("danceability").cast("double")) \
    .withColumn("valence", col("valence").cast("double"))

# Drop the first column (row index from CSV export)
df = df.drop(df.columns[0])

print(f"✓ Loaded {df.count():,} rows, {len(df.columns)} columns")
print(f"\nOptions that fixed it:")
print("  quote='"'      → Fields can be wrapped in double quotes")
print("  escape='\\'    → Quotes inside quotes are escaped as \"")
print("  multiline=true → A field can span multiple lines")
```

✓ Loaded 114,000 rows, 20 columns

Options that fixed it:

```
quote='"'      → Fields can be wrapped in double quotes
escape='\\'    → Quotes inside quotes are escaped as \"
multiline=true → A field can span multiple lines
```

```
In [10]: # Convert to RDD – should work now!
tracks_rdd = df.rdd

print(f"Type: {type(tracks_rdd)}")
print(f"Partitions: {tracks_rdd.getNumPartitions()}")

sample = tracks_rdd.first()
print(f"\n✓ Success! Sample track:")
print(f"  Track: {sample.track_name}")
print(f"  Artist: {sample.artists}")
print(f"  Genre: {sample.track_genre}")
print(f"  Energy: {sample.energy}")
```

Type: <class 'pyspark.rdd.RDD'>

Partitions: 1

✓ Success! Sample track:

Track: Comedy

Artist: Gen Hoshino

Genre: acoustic

Energy: 0.461

Lesson Learned: Real Data is Messy

What we just experienced is **normal** in data engineering:

Problem	Solution
Commas inside quoted fields	quote and escape options
Fields spanning multiple lines	multiLine=true
Some values still invalid	Explicit casting (bad values → NULL)

In production: Always validate data quality and log how many rows had issues.

2.2 Understanding the Lineage (DAG)

Every RDD knows its "recipe" — the chain of transformations that created it.

```
In [11]: # View the lineage
print(tracks_rdd.toDebugString().decode('utf-8'))
```

```
(1) MapPartitionsRDD[43] at javaToPython at NativeMethodAccessorImpl.java:0 []
| MapPartitionsRDD[42] at javaToPython at NativeMethodAccessorImpl.java:0 []
| SQLExecutionRDD[41] at javaToPython at NativeMethodAccessorImpl.java:0 []
| MapPartitionsRDD[40] at javaToPython at NativeMethodAccessorImpl.java:0 []
| FileScanRDD[39] at javaToPython at NativeMethodAccessorImpl.java:0 []
```

Reading the Debug String

```
(8) MapPartitionsRDD[5] at rdd at <ipython>:1 []
| MapPartitionsRDD[4] at rdd at <ipython>:1 []
| FileScanRDD[3] at csv at NativeMethodAccessorImpl.java:0 []
```

Part	Meaning
(8)	Number of partitions
MapPartitionsRDD	RDD type
[5]	RDD ID (the 5th RDD created in this session)
[]	Storage level — empty = NOT cached
\	Lineage arrow — parent RDD below

2.3 Discussion: Partitions vs Cores

Partitions vs Cores	Result
partitions < cores	Cores sit idle
partitions = cores	Fully utilized, but fragile
partitions = 2-4× cores	Sweet spot
partitions >> cores	Scheduling overhead

Why 2-4× cores is better than exactly equal:

- Tasks don't all take the same time (data skew)
- When a fast task finishes, the core can grab another partition
- Better load balancing

Rule of thumb: For small files, Spark may default to few partitions. You can repartition if needed: `rdd.repartition(16)`

Question 2

How many partitions does your RDD have? Is this a good number for your machine?

Your Answer: My RDD has 1 partition, but my machine has 16 cores. This is not a good number because:

- Only 1 core will be used — the other 15 cores sit idle
- There's no parallelism — Spark loses its main advantage
- Processing will be as slow as regular Python

Part 3: Lazy Evaluation

This is one of Spark's most important concepts. Watch the timing!

```
In [14]: # Define a chain of transformations
start = time.time()

high_energy = tracks_rdd \
    .filter(lambda row: row.energy is not None and row.energy > 0.8) \
    .filter(lambda row: row.popularity is not None and row.popularity > 50) \
    .map(lambda row: (row.track_name, row.artists, row.energy))

transform_time = time.time() - start
print(f"Transformations defined in: {transform_time:.4f} seconds")
print(f"\nHas any data been processed? NO! These are just transformations.")
```

Transformations defined in: 0.0010 seconds

Has any data been processed? NO! These are just transformations.

In []:

```
In [15]: # Now trigger an ACTION
start = time.time()

results = high_energy.take(10) # <-- ACTION!

action_time = time.time() - start
print(f"Action completed in: {action_time:.2f} seconds")
print(f"\nNOW the data was processed!")
print(f"\nHigh energy popular tracks:")
for track, artist, energy in results:
    print(f" {energy:.2f} | {track[:40]} - {artist[:20]}")
```

Action completed in: 1.49 seconds

NOW the data was processed!

High energy popular tracks:

0.91		透明だった世界	-	Motohiro Hata
0.89		The Enemy	-	Andrew Belle
0.91		Kaleidoscope	-	A Great Big World
0.84		Brave	-	Sara Bareilles
0.84		Heaven Knows	-	Five For Fighting
0.80		Kiss Me Slowly	-	Parachute
0.86		Without You	-	Parachute
0.83		Hard Sun	-	Eddie Vedder
0.89		Hyouriiittai	-	YUZU
0.93		Brand New	-	Ben Rector

Question 3

Why did the transformations take ~0.0001 seconds but `take(10)` took much longer?

Your Answer:

The transformations (filter, filter, map) took only 0.0010 seconds because Spark uses lazy evaluation — it doesn't actually process any data. It just records the operations in a DAG (execution plan). No CSV reading, no filtering, no mapping happens yet.

The take(10) action took 1.49 seconds because it's an action that triggers actual execution. At this point Spark:

- Reads 114,000 rows from the CSV file (disk I/O)
- Applies the first filter (energy > 0.8) to every row
- Applies the second filter (popularity > 50) to remaining rows
- Runs the map transformation to extract 3 fields
- Returns the first 10 results to the driver
- In short: Transformations = planning (instant), Actions = doing the work (takes time).

Part 4: Transformations vs Actions

Every RDD operation is either a **transformation** (lazy, returns RDD) or an **action** (eager, returns value).

```
In [16]: # Let's identify some operations
print("Operation          | Type")
print("-" * 33)

# filter() - returns an RDD
filtered = tracks_rdd.filter(lambda row: row.energy > 0.5)

print(f"filter()          | {type(filtered).__name__}")

# map() - returns an RDD
mapped = tracks_rdd.map(lambda row: row.track_name)
print(f"map()              | {type(mapped).__name__}")

# count() - returns a number
count = tracks_rdd.count()
print(f"count()            | {type(count).__name__} = {count:,}")

# take() - returns a List
taken = tracks_rdd.take(5)
print(f"take()              | {type(taken).__name__}")

# collect() - returns a List (DANGER: brings all data to driver!)
# collected = tracks_rdd.collect() # DON'T DO THIS on large datasets!
```

Operation	Type
filter()	PipelinedRDD
map()	PipelinedRDD
count()	int = 114,000
take()	list

🤔 Question 4

Which operations are transformations and which are actions? What's the simple rule to tell them apart?

Your Answer:

Transformations:

- filter() → Returns PipelinedRDD
- map() → Returns PipelinedRDD

Actions:

- count() → Returns int (114,000)
- take(5) → Returns list

The Simple Rule: Look at what the operation returns:

- Returns an RDD? → It's a Transformation (lazy, builds the plan)
- Returns a value (int, list, etc.)? → It's an Action (eager, executes the plan)

Transformations just define what to do; actions make Spark actually do it.

Part 5: Pair RDDs and Aggregation

Pair RDDs contain (key, value) tuples — essential for grouping and aggregating data.

Let's find the **average energy by genre**.

```
In [17]: # Step 1: Create Pair RDD of (genre, energy)
genre_energy = tracks_rdd \
    .filter(lambda row: row.track_genre is not None and row.energy is not None) \
    .map(lambda row: (row.track_genre, row.energy))

genre_energy.take(5)
```

```
Out[17]: [('acoustic', 0.461),
          ('acoustic', 0.166),
          ('acoustic', 0.359),
          ('acoustic', 0.0596),
          ('acoustic', 0.443)]
```

```
In [18]: # Step 2: Calculate average energy per genre
# Strategy: (genre, energy) -> (genre, (sum, count)) -> (genre, average)

genre_avg_energy = genre_energy \
    .mapValues(lambda energy: (energy, 1)) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .mapValues(lambda x: x[0] / x[1])

# Sort by energy (descending) and show top 15
high_energy_genres = genre_avg_energy \
    .sortBy(lambda x: -x[1]) \
    .take(15)

print("Top 15 Highest Energy Genres:")
print("-" * 35)
for genre, avg_energy in high_energy_genres:
    print(f"{genre:25} | {avg_energy:.3f}")
```

Top 15 Highest Energy Genres:

```
-----
death-metal          | 0.931
grindcore            | 0.924
metalcore            | 0.914
happy                | 0.911
hardstyle            | 0.901
drum-and-bass        | 0.877
black-metal          | 0.875
heavy-metal          | 0.874
party                | 0.871
j-idol               | 0.869
industrial            | 0.862
breakbeat            | 0.853
trance               | 0.845
hardcore             | 0.842
metal                | 0.840
```

Question 5

Now write a query for the most chill genres, ie those with the 15 lowest energy and execute it.

Your answer:

```
In [21]: # What about the most chill genres?

# YOUR CODER HERE
chill_genres = genre_energy \
    .mapValues(lambda energy: (energy, 1)) \
```

```

    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .mapValues(lambda x: x[0] / x[1]) \
    .sortBy(lambda x: x[1]) \
    .take(15)

print("Top 15 Most Chill (Lowest Energy) Genres:")
print("-" * 35)
for genre, avg_energy in chill_genres:
    print(f"{genre:25} | {avg_energy:.3f}")

```

Top 15 Most Chill (Lowest Energy) Genres:

```

-----
classical          | 0.190
new-age           | 0.215
ambient           | 0.237
romance           | 0.294
disney            | 0.303
opera             | 0.317
piano             | 0.320
guitar            | 0.325
sleep             | 0.342
jazz              | 0.353
honky-tonk        | 0.367
tango             | 0.373
show-tunes        | 0.399
study             | 0.411
chill             | 0.427

```

Why `reduceByKey` instead of `groupByKey` ?

We could have written:

```

# DON'T DO THIS
genre_energy.groupByKey().mapValues(lambda vals:
    sum(vals)/len(list(vals)))

```

But `reduceByKey` is **much** more efficient:

<code>groupByKey</code>	<code>reduceByKey</code>
Shuffles ALL values across network	Combines locally FIRST, then shuffles
114K records transferred	Only ~114 partial sums transferred
Can cause out-of-memory	Memory efficient
Almost never the right choice	Use this for aggregations

Question 6

In your own words, why does `reduceByKey` transfer less data than `groupByKey` ?

Your Answer:

reduceByKey transfers less data than groupByKey because it combines values locally on each machine before shuffling.

With groupByKey, ALL 114,000 energy values must travel across the network to be grouped. With reduceByKey, each machine first computes partial (sum, count) tuples locally, then only these ~114 partial results (one per genre) are shuffled. This is like counting votes: instead of sending every ballot to headquarters, each precinct counts locally and sends only the totals.

Part 6: The Caching Experiment

What happens when we use the same RDD multiple times?

```
In [22]: # Create a filtered RDD (NOT cached)
popular_rdd = tracks_rdd.filter(lambda row: row.popularity is not None and row.popularity > 70)

print("WITHOUT cache():")
print("-" * 40)

start = time.time()
count1 = popular_rdd.count()
time1 = time.time() - start
print(f"First count(): {count1:,} tracks in {time1:.3f}s")

start = time.time()
count2 = popular_rdd.count()
time2 = time.time() - start
print(f"Second count(): {count2:,} tracks in {time2:.3f}s")

print(f"\nTotal time: {time1 + time2:.3f}s")
```

WITHOUT cache():

First count(): 4,846 tracks in 2.945s

Second count(): 4,846 tracks in 2.674s

Total time: 5.619s

Question 7

Now based on the lecture modify the above query such that you cache the RDD you are building appropriately.

Your answer:

```
In [27]: # Now WITH cache()

# YOUR CODE HERE
popular_cached = tracks_rdd \
    .filter(lambda row: row.popularity is not None and row.popularity > 70) \
    .cache() # Cache the RDD in memory
```

```

# First execution (loads into cache)
start = time.time()
count1 = popular_cached.count()
time1 = time.time() - start
print(f"First count(): {count1:,} tracks in {time1:.3f}s")

# Second execution (reads from cache)
start = time.time()
count2 = popular_cached.count()
time2 = time.time() - start
print(f"Second count(): {count2:,} tracks in {time2:.3f}s")

print(f"\nTotal time: {time1 + time2:.3f}s")
if time2 > 0:
    print(f"Speedup: {time1/time2:.1f}x faster!")

```

First count(): 4,846 tracks in 4.210s

Second count(): 4,846 tracks in 1.411s

Total time: 5.621s

Speedup: 3.0x faster!

In [25]:

```
# Check the Lineage – now shows storage Level
print(popular_cached.toDebugString().decode('utf-8'))
```

```

(1) PythonRDD[76] at RDD at PythonRDD.scala:53 [Memory Serialized 1x Replicated]
    |      CachedPartitions: 1; MemorySize: 602.9 KiB; DiskSize: 0.0 B
    |      MapPartitionsRDD[43] at javaToPython at NativeMethodAccessorImpl.java:0 [Memory Serialized 1x Replicated]
    |      MapPartitionsRDD[42] at javaToPython at NativeMethodAccessorImpl.java:0 [Memory Serialized 1x Replicated]
    |      SQLExecutionRDD[41] at javaToPython at NativeMethodAccessorImpl.java:0 [Memory Serialized 1x Replicated]
    |      MapPartitionsRDD[40] at javaToPython at NativeMethodAccessorImpl.java:0 [Memory Serialized 1x Replicated]
    |      FileScanRDD[39] at javaToPython at NativeMethodAccessorImpl.java:0 [Memory Serialized 1x Replicated]

```

Question 8

How much faster was the second `count()` with caching? When would you use `cache()` ?

Your Answer:

The second count() was approximately 3.0x faster with caching.

I would use cache() when:

- Reusing an RDD multiple times – each action would otherwise recompute from scratch
- Expensive computations – complex transformations, joins, or reading from slow storage
- Iterative algorithms – like machine learning where the same data

is used in every iteration

- Interactive exploration – when running multiple queries on the same filtered dataset

The trade-off is memory usage — the cached data stays in RAM, so only cache what you actually reuse and call `unpersist()` when done

Part 7: Comparing RDD vs DataFrame

Let's do the same genre analysis with DataFrames to see the difference.

```
In [31]: # RDD way (what we did):
tracks_rdd.map(lambda row: (row.track_genre, row.energy)) \
    .mapValues(lambda e: (e, 1)) \
    .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])) \
    .mapValues(lambda x: x[0]/x[1]) \
    .sortBy(lambda x: -x[1]).take(10)

# DataFrame way:
# df.groupBy("track_genre") \
#     .avg("energy") \
#     .orderBy("avg(energy)", ascending=False) \
#     .show(10)
```

```
Out[31]: [('death-metal', 0.9314700000000008),
('grindcore', 0.9242010000000016),
('metalcore', 0.9144849999999995),
('happy', 0.9109710000000006),
('hardstyle', 0.9012460000000007),
('drum-and-bass', 0.8766350000000008),
('black-metal', 0.8748973000000019),
('heavy-metal', 0.8740029999999996),
('party', 0.8712370000000005),
('j-idol', 0.8686772999999997)]
```

```
In [33]: # RDD way (what we did):
# tracks_rdd.map(lambda row: (row.track_genre, row.energy)) \
#     .mapValues(lambda e: (e, 1)) \
#     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])) \
#     .mapValues(lambda x: x[0]/x[1]) \
#     .sortBy(lambda x: -x[1]).take(10)

# DataFrame way:
df.groupBy("track_genre") \
    .avg("energy") \
    .orderBy("avg(energy)", ascending=False) \
    .show(10)
```

```
+-----+-----+
| track_genre|      avg(energy)|
+-----+-----+
| death-metal|0.9314700000000008|
| grindcore|0.9242010000000016|
| metalcore|0.9144849999999995|
| happy|0.9109710000000006|
| hardstyle|0.9012460000000007|
| drum-and-bass|0.8766350000000008|
| black-metal|0.8748973000000019|
| heavy-metal|0.8740029999999996|
| party|0.8712370000000005|
| j-idol|0.8686772999999997|
+-----+-----+
only showing top 10 rows
```

Same result, but DataFrame is:

- More readable
- Optimized by Spark's Catalyst engine
- Less error-prone

So why learn RDDs?

- DataFrames compile down to RDD operations
- Understanding partitions, shuffles, and caching applies to both
- Some operations still need RDDs (custom aggregations, complex logic)
- Interview questions

Part 8: Your Turn!

Write Spark code to answer TWO of these questions:

1. **Which artist has the most tracks in the dataset?** (Pair RDD with artist as key)
2. **What's the average danceability of tracks with popularity > 80?**
3. **Which genre has the happiest music?** (highest average valence)
4. **Find "sad bangers"** — tracks that are highly danceable (>0.8) but sad (valence <0.3)

Write your code below:

```
In [35]: # YOUR CODE HERE
# Access fields by name: row.track_name, row.artists, row.track_genre,
#                      row.popularity, row.danceability, row.energy, row.valence

# =====
# EXERCISE 1: Which artist has the most tracks?
# =====
print("=" * 60)
print("EXERCISE 1: Artist with Most Tracks")
```



```

print("=" * 60)

top_artists = tracks_rdd \
    .filter(lambda row: row.artists is not None) \
    .map(lambda row: (row.artists, 1)) \
    .reduceByKey(lambda a, b: a + b) \
    .sortBy(lambda x: -x[1]) \
    .take(10)

print("\nTop 10 Artists by Track Count:")
print("-" * 45)
for artist, count in top_artists:
    print(f"{artist[:35]:<35} | {count:>5} tracks")

# =====
# EXERCISE 3: Which genre has the happiest music?
# =====
print("\n" + "=" * 60)
print("EXERCISE 3: Happiest Genres (Highest Average Valence)")
print("=" * 60)

happiest_genres = tracks_rdd \
    .filter(lambda row: row.track_genre is not None and row.valence is not None) \
    .map(lambda row: (row.track_genre, row.valence)) \
    .mapValues(lambda v: (v, 1)) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .mapValues(lambda x: x[0] / x[1]) \
    .sortBy(lambda x: -x[1]) \
    .take(15)

print("\nTop 15 Happiest Genres:")
print("-" * 45)
for genre, avg_valence in happiest_genres:
    mood = "😊" if avg_valence > 0.7 else ("😐" if avg_valence > 0.5 else "😞")
    print(f"{genre:<25} | {avg_valence:.3f} {mood}")

```

```
=====
EXERCISE 1: Artist with Most Tracks
=====
```

Top 10 Artists by Track Count:

```
-----
The Beatles           | 279 tracks
George Jones          | 271 tracks
Stevie Wonder         | 236 tracks
Linkin Park           | 224 tracks
Ella Fitzgerald       | 222 tracks
Prateek Kuhad         | 217 tracks
Feid                  | 202 tracks
Chuck Berry           | 190 tracks
Håkan Hellström       | 183 tracks
OneRepublic           | 181 tracks
```

```
=====
EXERCISE 3: Happiest Genres (Highest Average Valence)
=====
```

Top 15 Happiest Genres:

```
-----
salsa                 | 0.815 😄
forro                 | 0.761 😄
rockabilly           | 0.727 😄
afrobeat             | 0.699 😊
ska                  | 0.697 😊
children             | 0.694 😊
samba                | 0.693 😊
pagode               | 0.688 😊
party                | 0.681 😊
kids                 | 0.681 😊
disco                | 0.671 😊
rock-n-roll          | 0.670 😊
reggae               | 0.648 😊
reggaeton            | 0.643 😊
bluegrass            | 0.635 😊
```

Part 9: Cleanup

Always stop Spark when done!

```
In [36]: # Unpersist cached RDDs and stop Spark
popular_cached.unpersist()
spark.stop()
print("✓ Spark stopped. Lab complete!")
```

✓ Spark stopped. Lab complete!

Summary

Concept	What You Learned
Partitions	Data split across cores; 2-4× cores is ideal
Lazy Evaluation	Transformations build a plan; actions execute it
Transformations	<code>filter()</code> , <code>map()</code> , <code>mapValues()</code> , <code>sortBy()</code> — return RDDs
Actions	<code>count()</code> , <code>take()</code> , <code>collect()</code> — return values
Pair RDDs	<code>(key, value)</code> tuples for grouping
reduceByKey	Combines locally first — much faster than <code>groupByKey</code>
Caching	<code>cache()</code> stores RDD in memory for reuse

The concepts you learned apply whether you use RDDs or DataFrames — they're the foundation of Spark!